

Culture Scientifique - Introduction à la pensée algorithmique

6 janvier 2025

Ce cours a pour objectif de sensibiliser aux notions d'algorithmique présente dans la vie de tous les jours, mais principalement dans les programmes de mathématiques, et de sciences dans les cycles 3 et 4, ainsi qu'au concours. Pour cela, un certain pré-requis mathématique est nécessaire : numération, calcul, calcul littéral, géométrie élémentaire de niveau collège.

On se fixe la liste d'attendus suivants :

1. Maîtriser la notion d'**instruction élémentaire**, et le jeu d'instructions proposé par Scratch : déclaration de variable, affectation, structure conditionnelle, structure itérative ;
2. Repérer des suites d'instructions élémentaires dans la vie de tous les jours, en mathématiques, en physique, en informatique ;
3. Connaître les éléments constitutifs d'un ordinateur et leur rôle ;
4. Être capable de lire un algorithme simple, ou un programme simple écrit en Scratch ;
5. Traduire des instructions en langage naturel vers le langage Scratch ;
6. Utiliser un tableur pour résoudre un problème (formules, fonctions simples) ;
7. Exploiter les notions mathématiques pour répondre à des problèmes.

On profitera du fait que ce cours soit plutôt orienté sur les outils informatiques pour s'intéresser également à l'utilisation du tableur, qui fait partie des attendus du CRPE.

Sources intéressantes :

- Exo7, Scratch au collège (<http://exo7.emath.fr/cours/livre-scratch.pdf>)
- Programme de mathématiques, sciences et technologie, cycles 3 et 4
- *Maths et numérique*, INSPE Réunion ¹

1 Vocabulaire introductif

Définition 1.1: Algorithme

Un algorithme est une succession d'opérations élémentaires permettant de donner une solution à un problème donné.

1. https://inspe.univ-reunion.fr/fileadmin/Fichiers/ESPE/disciplines/Mathematiques_PE/Ressources/T1.pdf

Un algorithme a en général besoin de données pour fonctionner, les données d'entrée, et, lorsqu'il se termine, retourne à l'utilisateur un résultat, les données de sortie.

Définition 1.2: Terminaison

Un algorithme se **termine** lorsqu'il n'y a plus d'instructions à exécuter.

On s'attend en général à ce que les dernières instructions soient les instructions communiquant le résultat à l'utilisateur.

Exemples d'algorithmes que vous connaissez (peut-être sans le savoir) :

- Résolution d'un calcul posé : il prend en entrée deux nombres, et retourne un nombre ;
- Le rendu de monnaie : il prend en entrée deux nombres, et retourne la liste des pièces à rendre ;
- Le tri de cartes à jouer, par valeur, ou par couleur : il prend en entrée un tas de cartes, et retourne le tas de cartes trié.

Pour le premier exemple, lors d'une addition posée, un exemple d'opération élémentaire est la somme de deux valeurs, ou alors le calcul de la valeur de la retenue.

Définition 1.3: Opérations élémentaires

En algorithmique, les opérations élémentaires principales sont les suivantes :

- Effectuer un calcul (addition, soustraction, multiplication, division, division euclidienne) ;
- **Lire** du texte rentré par l'utilisateur ;
- **Écrire** du texte, ou le contenu d'une variable ;
- Réaliser une **affectation** (Garder en mémoire une donnée) ;
- Récupérer une donnée en mémoire.

Exemple : lorsqu'on effectue une addition posée, la retenue est une donnée qu'on garde en mémoire (on l'inscrit sur la feuille), car on doit l'utiliser pour le prochain calcul.

$$\begin{array}{r}
 ^1 ^1 \\
 3892 \\
 + 5185 \\
 \hline
 9077
 \end{array}$$

FIGURE 1 – Addition faisant intervenir deux fois une retenue.

Exercice 1 — Décorticage

Détailler les étapes des algorithmes ci-dessus. Ces étapes doivent être élémentaires.

Corrigé de l'exercice 1

1. On se place sur la dernière colonne.
2. On additionne les deux chiffres. Le chiffre des unités est inscrit sous la barre, et si le chiffre des dizaines vaut au moins 1, alors on l'inscrit juste au dessus.

3. On recommence l'étape précédente, et s'il y a une retenue, alors on l'additionne avec le résultat. On continue **jusqu'à** ce qu'on atteigne la dernière colonne.

Définition 1.4: Variables

En algorithmique, les valeurs gardées en mémoire sont appelées des **variables**. Chaque variable est définie par quatre informations :

- Un rôle ;
- Un nom, qui permet d'identifier la variable. ;
- Un type, qui va indiquer à l'ordinateur ce que va contenir la variable ;
- Une valeur, l'information contenue au sein de la variable. Elle n'existe que pendant l'exécution de l'algorithme.

Une variable peut être qualifiée de **constante** : après initialisation, elle ne peut plus être modifiée pendant l'exécution.

Par convention, les noms de variable commencent par une minuscule, et les éventuels espaces sont représentés avec le tiret du bas `_`, les constantes s'écrivent en majuscules et sont déclarées au départ. Les constantes *universelles* disponibles et répandues dans tous les langages de programmation sont les constantes mathématiques comme π et e . Selon le problème traité par un algorithme, il peut être nécessaire d'utiliser des constantes. Par exemple, des constantes utilisées en physique (g), la taille maximale d'un tableau, les dimensions d'un jeu de plateau, la taille maximale d'un message, etc.

Définition 1.5: Types principaux

Voici les types principaux de données qui sont traitées par un algorithme :

- le nombre (entier, réel à virgule flottante) ;
- le caractère (une lettre, un chiffre ou un symbole) ;
- la chaîne de caractères (plusieurs caractères) ;
- la collection de plusieurs éléments de même type ;
- le booléen (Vrai ou Faux).

2 La syntaxe des instructions

Les instructions sont repérées par des **mots-clefs**, et peuvent nécessiter des **paramètres** pour fonctionner. Tout comme dans le langage naturel, il y a un ensemble de règles à respecter scrupuleusement, pour que les interlocuteurs puissent se comprendre². Nous allons donc proposer un jeu d'instructions, en les nommant et en décrivant leur fonctionnement : on dira que c'est un langage algorithmique **pseudo-code**, car on va essayer de s'approcher de la syntaxe des langages informatiques les plus populaires comme Python, Java, Javascript, C. On en profitera pour également voir la composition des blocs de Scratch.

2. Une seule erreur dans un programme provoque son extinction prématurée

2.1 Instructions d'entrée-sortie

On va appeler *Lire(valeur)* et *Ecrire("texte")* respectivement l'instruction qui permet de lire des données et de la placer dans la variable *valeur*, et *Ecrire* l'instruction qui permet d'écrire du texte, ou le contenu d'une variable, sur l'écran. Ici par exemple, *valeur* est un **paramètre** de l'instruction *Lire*. Pour ces deux instructions, on mettra les paramètres entre parenthèses, et séparés par une virgule (syntaxe utilisée par beaucoup de langages de programmation). La syntaxe est la suivante :

```
Lire(<variable>)
Ecrire(<valeur1>,<valeur2>,...)
```

Pour distinguer le cas où on écrit du texte du cas où on veut écrire le contenu d'une variable, on place le texte entre guillemets.

```
Ecrire("texte") # va afficher texte à l'écran
Ecrire "texte" # est incorrectement écrit d'après notre règle
```

L'instruction d'affichage sera comme indiqué dans l'exemple ci-dessous avec la variable *texte* :

```
texte <- "Bonjour"
Ecrire(texte) # va écrire Bonjour
```

Il est également possible d'afficher plusieurs informations à la suite. On séparera chacune d'entre-elles par une virgule.

```
Ecrire("Bonjour, vous avez écrit :", texte)
```

Exercice 2 — Application de la TVA

Écrire un algorithme où l'utilisateur doit renseigner un prix en euros, et la machine doit afficher le prix en ayant appliqué une TVA de 20%. Puis implémenter sur Scratch cet algorithme.

On donnera exceptionnellement la correction prématurément pour illustrer le travail demandé. Comme indiqué dans l'énoncé, il faut une information d'entrée, qui est chargée dans la variable *prix*, et l'information de sortie est le prix augmenté de 20%, qu'on peut écrire directement dans la fonction d'affichage *Ecrire*.

Algorithme 1 : Application TVA

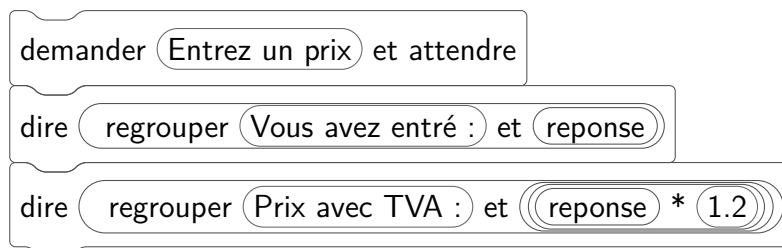
Données : Un nombre *prix*

Résultat : Le prix avec la TVA

- 1 Lire(*prix*) ;
 - 2 Ecrire("Vous avez rentré :", *prix*);
 - 3 Ecrire("Prix avec TVA :", $prix \times 1,2$);
-

Dans Scratch, le bloc **Demander** stocke automatiquement l'entrée utilisateur dans une variable **réponse**, qu'on peut réutiliser à la suite. Le bloc **regrouper** permet de concaténer deux chaînes de caractères.

L'utilisateur écrit du texte, et Scratch convertit automatiquement le contenu dans le type correspondant. Si on écrit une chaîne de caractères qui n'est pas la représentation d'un nombre, comme *aaa*, alors si on veut effectuer un calcul avec, elle aura une valeur de 0.



2.2 Les structures conditionnelles

Les structures conditionnelles permettent d'exécuter un bloc d'instructions en fonction du résultat à un **test logique**. On l'appelle aussi **structure si-alors**, en référence à sa syntaxe.

Définition 2.1: Test logique

Un test logique est une **opération** qui retourne une valeur **logique** : **vrai** ou **faux**.

Opérateurs de comparaison	Signification
=	test d'égalité
!=	test de différence
>=	supérieur ou égal
>	supérieur strict
<=	inférieur ou égal
<	inférieur strict

En algorithmique, on **déclare** une structure conditionnelle avec le mot-clef **Si**, suivi du test à réaliser, puis on indique le bloc à réaliser si le test renvoie **Vrai** avec le mot-clef **Alors**, et si c'est nécessaire, on indique le bloc d'instructions à réaliser lorsque le test renvoie **Faux** avec le mot-clef **Sinon**.

```

# est-ce que le reste de a divisé par 2 est nul ?
Si a%2 = 0 alors
    a <- a//2
Sinon
    a <- 3*a + 1
  
```

Définition 2.2: Opérateurs logiques

Les opérateurs logiques permettent de composer des tests logiques plus avancés. On en donne trois :

- Le **ou logique**, qui renvoie vrai si au moins un des tests est vrai ;
- Le **et logique**, qui renvoie vrai si tous les tests sont vrais ;
- Le **non logique**, qui renvoie le contraire du test donné.

Exercice 3 — Inégalité triangulaire

On donne les longueurs a , b et c de trois segments. Écrire un bloc d'instructions qui indique si ces trois segments peuvent former un triangle. Le retour se fera avec du texte affiché à l'écran.

Indication : **au moins un** des côtés doit être supérieur à la somme des deux autres.

Exercice 4 — Souvenirs de Pythagore

On donne les longueurs a , b et c des côtés d'un triangle. Ecrire un programme qui indique s'il est rectangle, ainsi que le nom de l'hypoténuse.

Exercice 5 — Mention ?

A partir d'une note entre 0 et 20, créer un algorithme qui indique la mention obtenue au diplôme : passable à partir de 10, AB à partir de 12, B à partir de 14 et TB à partir de 16.

Pour faire ces exercices sur Scratch, il faudra au préalable déclarer 3 variables, et après avoir rentré une valeur au clavier, on l'affectera à une de ces variables.

2.3 Blocs personnalisés

Un bloc d'instructions n'est rien d'autre qu'une série d'instructions donnée. Si jamais un bloc est présent plusieurs fois dans un algorithme, alors on donnera un nom qui fera référence à ces instructions, ce qui permettra de les écrire une fois, mais de les exécuter autant de fois qu'on veut. On appellera cette structure une **routine**.

Définition 2.3: Routines

*Une routine est une série d'instructions élémentaires, dont on peut spécifier les données d'entrée, les **paramètres**, et éventuellement, les valeurs retournées.*

On donne en algorithmique la syntaxe simplifiée suivante :

```
Routine maximum3nombres(a,b,c)
  Si (a >= b) et (a >= c) alors
    retourner a
  Sinon si (b >= a) et (b >= c) alors
    retourner b
  Sinon
    retourner c
```

La valeur retournée peut ensuite être réutilisée dans une autre instruction, ou être affectée à une variable :

```
Ecrire(maximum(5,7,3)) #affichera donc 7
```

Malheureusement, Scratch ne permet pas de retourner de valeur avec un bloc personnalisé, ce qui est quand même cocasse pour un logiciel qui se veut être langage de programmation. Il faudra faire avec ! Ainsi, dans la figure 2, la routine **entrer_abc** effectue les entrées utilisateur, tandis que la routine **maximum** réalise les instructions pour les trois valeurs a , b et c , ce qui résume le programme principal à deux blocs.

Attention : les paramètres a , b et c ne sont pas les variables a , b et c qu'on a déclarées précédemment, mais des variables temporaires qui ne sont utilisées que le temps de la routine^a. On voit en effet que les couleurs utilisées sont différentes.

^a. Il semblerait que Scratch ne permette même pas de les modifier...

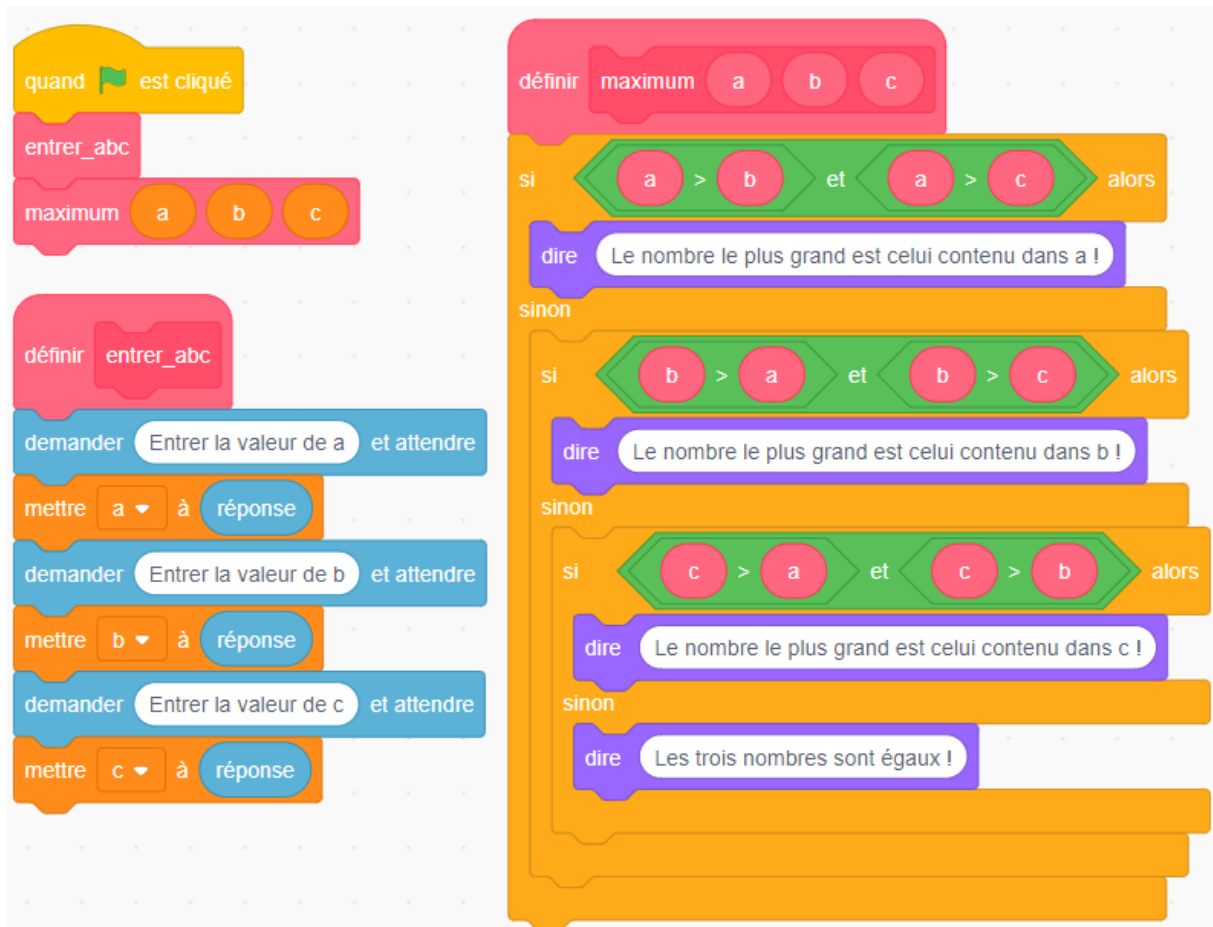


FIGURE 2 – Le programme du maximum de trois nombres, en séparant les étapes dans deux routines `entrer_abc` et `maximum(a,b,c)`

2.4 Les structures itératives

Définition 2.4: Structures itératives

Une structure itérative consiste à **répéter** un nombre de fois défini à l'avance, ou **tant qu'** une condition logique est vérifiée.

On donne en algorithmique deux types de boucles : les boucles **Pour** et **Tant que**.

2.4.1 Boucle Pour

Algorithme 2 : Somme des n premiers entiers avec boucle pour

Données : Deux entiers n et $total$

Résultat : La somme des entiers de 1 à n

```

1  $total \leftarrow 0$  ;
2 Lire( $n$ ) ;
3 pour  $i$  allant de 1 à  $n$  par pas de 1 faire
4   |  $total \leftarrow total + i$ ;
5 fin
6 Ecrire( $total$ ) ;
```

La boucle Pour se déclare avec une **variable de boucle**, une valeur de départ et une valeur d'arrivée, ainsi que le pas. Ainsi, dans l'algorithme ci-dessus, en entrant dans la boucle, i prend la valeur 1, et on ajoute sa valeur à la variable $total$, puis on passe à la prochaine valeur de i qui est 2, et on recommence.

Définition 2.5: Trace

Une **trace** consiste à consigner l'état de l'algorithme ou du programme après avoir exécuté une instruction, ou une itération de boucle.

On donne ci-dessous la trace de l'algorithme précédent, en donnant 4 pour valeur de n . On voit clairement ici que la valeur de n n'est jamais modifiée. Avant de rentrer dans la boucle, la variable i ne contient **pas de valeur**.

Itérations	total	n	i
0	0	4	/
1	1	4	1
2	3	4	2
3	6	4	3
4	10	4	4

2.4.2 Boucle Tant que

La boucle **Tant que** se déclare avec une condition logique. Si cette condition logique est vérifiée, alors on exécute les instructions du bloc et on recommence, sinon, on sort du bloc. Le cas typique et scolaire d'utilisation d'une boucle **Tant que** est la recherche de seuil. Exemple : On stocke 10000 euros dans un compte bancaire avec un taux d'intérêts cumulés de 8% par an. Après combien d'années la somme contenue à l'intérieur aura doublé ?

Il est tout à faire possible de donner une réponse *à la main* : on cherche un nombre entier d'années n tel que $10000 \times 1,08^n \geq 20000$:

$$\begin{aligned} 10000 \times 1,08^n &\geq 20000 \\ 1,08^n &\geq 2 \\ n &\geq \frac{\ln 2}{\ln 1,08} \\ n &\geq 9.006 \end{aligned}$$

Et on conclut donc que $n = 10$ en arrondissant à l'entier supérieur, cependant, cela a nécessité l'utilisation du logarithme népérien, qui n'est pas attendu du programme du CRPE.

Algorithme 3 : Recherche de seuil

Données : Un entier n et trois nombres réels *compte*, *taux* et *somme_initiale*

Résultat : Le nombre d'années n pour que le contenu du compte soit doublé

```

1  $n <- 0$  ;
2 Lire(somme_initiale) ;
3 Lire(taux) ;
4 compte  $<-$  somme_initiale ;
5 tant que compte  $<$   $2 \times$  somme_initiale faire
6   |  $n <- n + 1$ ;
7   | compte  $<-$  compte  $\times$   $(1 + \textit{taux})$ 
8 fin
9 Ecrire(Il faudra  $n$  ans pour doubler la somme) ;
```

Ainsi, nous demandons à l'ordinateur de multiplier par 1,08 le contenu du compte et ce, **tant que** celle-ci est inférieure au double de la somme initiale. Cette opération est également facilement réalisable sur un tableur à l'aide de la poignée de recopie (voir figure 3). Dans la figure 3, on précise que la cellule B2 affiche 8%, mais le nombre contenu à l'intérieur est 0,08 ; la formule inscrite en E3, $E2 \times (1 + B\$2)$ est donc bien correct.

Exercice 6 — Polygones réguliers

Créer une routine qui prend deux paramètres : un nombre de côtés et la longueur d'un côté, et qui trace le polygone régulier correspondant. On utilisera donc une structure itérative, où on va répéter le mouvement du lutin, puis une rotation de l'**angle externe** pour qu'il suive le tracé correctement.

E3 $\text{fx} = \text{E2} * (1 + \text{B\$2})$

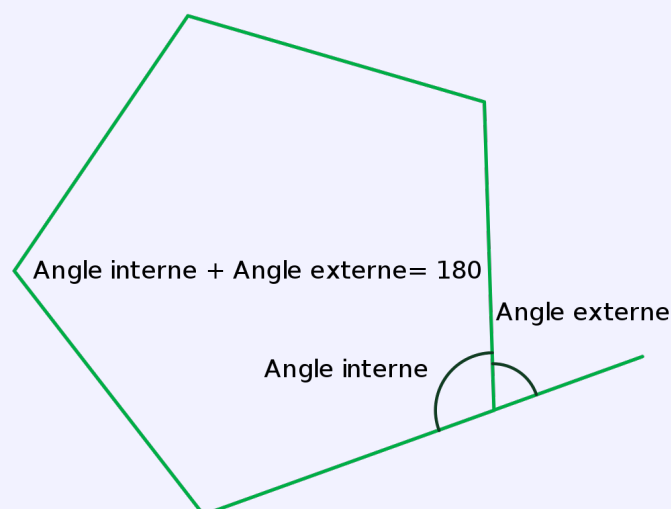
	A	B	C	D	E
1	Somme initiale	10000		années	somme
2	Taux	8,00%		0	10000
3				1	10800
4				2	11664
5				3	12597,12
6				4	13604,8896
7				5	14693,28077
8				6	15868,74323
9				7	17138,24269
10				8	18509,3021
11				9	19990,04627
12				10	21589,24997

FIGURE 3 – Réalisation de la recherche de seuil avec un tableur.

Définition 2.6: Angles internes et externes

Un angle interne d'un polygone est formé en un sommet, et situé à l'intérieur, et en prolongeant un côté, l'angle formé par ce côté prolongé et le côté adjacent est appelé angle externe. Pour un polygone régulier à n côtés :

- l'angle interne vaut $\frac{n-2}{n} \times 180$ degrés ;
- l'angle externe vaut $\frac{360}{n}$ degrés .



Pour cet exercice, il est nécessaire de bien comprendre le fonctionnement du **lutin**.

Définition 2.7: Lutin

Dans Scratch, un lutin est un objet caractérisé par plusieurs attributs :

- Son image, ou *sprite* ;
- Sa position dans le canevas, avec deux coordonnées x et y . L'axe des abscisses est orienté vers la droite, l'axe des ordonnées vers le haut ;
- Son orientation **en degrés**. Dans Scratch, un angle est compris dans l'intervalle $] -180; 180]$:
 - S'orienter à 0 degré veut dire qu'on s'oriente vers le haut ;
 - S'orienter à 90 degrés veut dire qu'on s'oriente vers la droite ;
 - S'orienter à 180 degrés veut dire qu'on s'oriente vers le bas ;
 - S'orienter à 270 degrés (équivalent à -90 degrés) veut dire qu'on s'oriente vers la gauche ;
- l'état de son stylo (posé ou levé). Si un lutin se déplace alors que son stylo est posé, alors il tracera une ligne sur son trajet.

Chaque lutin dispose de ses propres instructions, et ces instructions s'exécutent en parallèle.

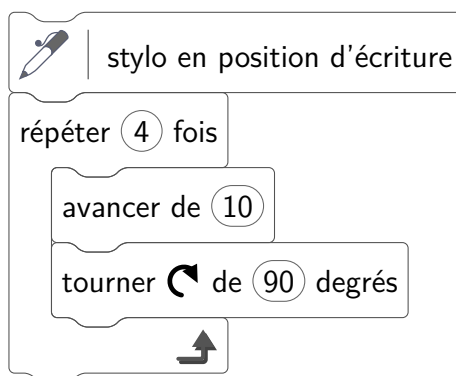
Dans une optique de vérification, mais également de *debug* (programme fonctionnant incorrectement), on peut être amené à réaliser une **trace**, pas à pas, des instructions, et de consulter à chaque fois l'**état** du programme.

3 Exercices guidés

Nous présentons ici plusieurs types d'exercices attendus à partir des connaissances ci-dessus.

Exercice 7 — Réaliser une trace tableau

On considère que le lutin est en $(15; -5)$ et est orienté de 90 degrés vers la droite au départ. Que va faire le lutin dans ce programme ? Donner après chaque itération sa position ainsi que son orientation.



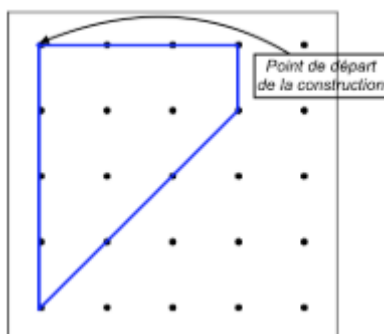
Voici un exemple de présentation de trace pour cet exercice, où on met en valeur les informations exploitées par le programme :

Itérations	x	y	orientation
0	15	-5	90
1	25	-5	180
2	25	-15	270 (-90)
3	15	-15	360 (0)
4	15	-5	450 (90)

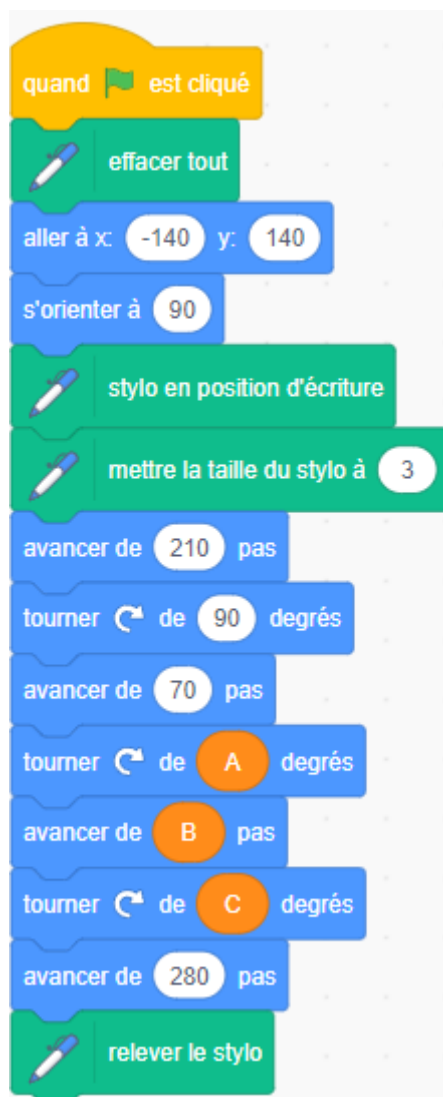
Exercice 8 — Compléter un programme / Trace visuelle

Vu dans CRPE 2024 Groupement 1

Un programme réalisé sur le logiciel « Scratch » permet de construire la figure ci-dessous.



En prenant 3 cm pour 70 pas, déterminer les valeurs à attribuer aux lettres A, B et C pour que le script proposé ci-contre permette de construire cette figure. On prendra une valeur approchée à l'unité pour B. Le point de départ de la figure a pour coordonnées (140 ; 140). On rappelle que *s'orienter à 90* signifie *s'orienter vers la droite*.



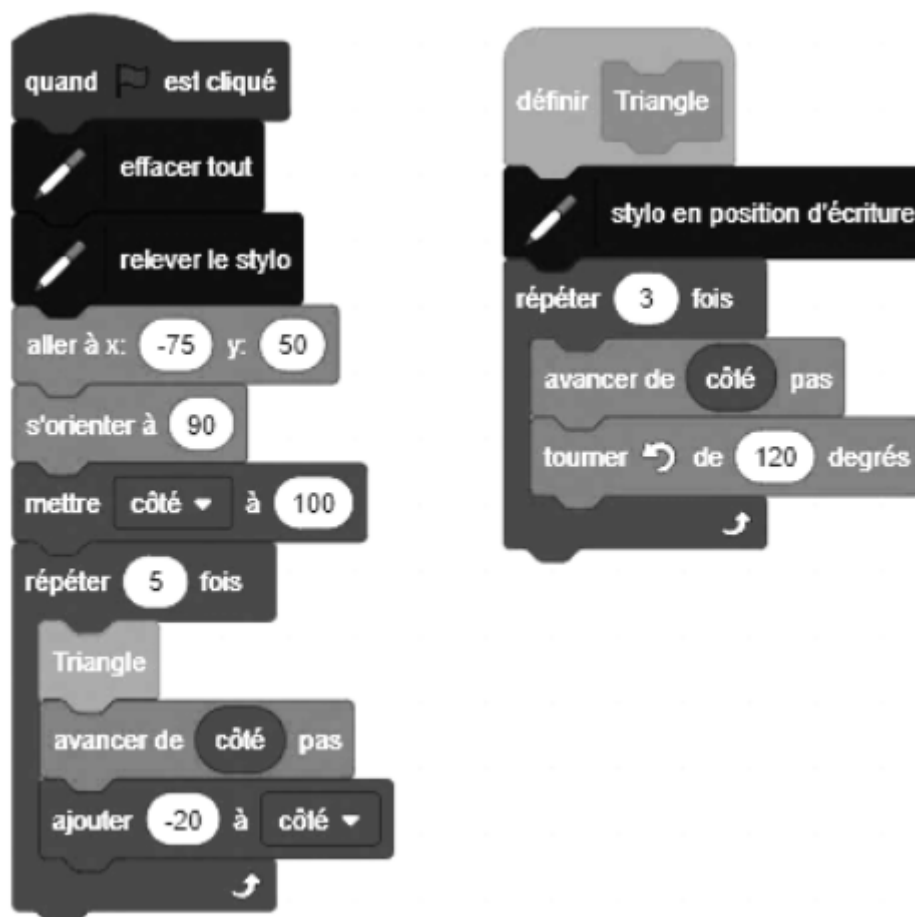
Corrigé de l'exercice 8

Il est intéressant pour cet exercice de reproduire la figure en prolongeant les côtés pour comprendre les angles demandés dans cet exercice.

En tournant pour la première fois, on voit qu'on trace la diagonale d'un carré, qui coupe l'angle droit en deux. Il faut donc tourner de 45 degrés vers la droite, donc A vaudra 45. Ce carré a une longueur de 9 cm, soit 210 pas. On rappelle que la diagonale d'un carré de côté c vaut $c\sqrt{2}$ (qu'on peut retrouver avec Pythagore, mais on peut donner directement sa valeur). $210\sqrt{2} \approx 296.98 \approx 297$ (valeur approchée à l'unité comme demandé). Donc B vaudra 297. Enfin, on tourne à nouveau à droite selon la diagonale, mais ici, il faudra prendre la valeur de l'angle **supplémentaire** : $180 - 45 = 135$. Donc C vaudra 135.

Exercice 9 — Analyse générale de programme

Vu dans CRPE 2023 Groupement 1



1. Quelles sont les coordonnées du point de départ ?
2. Combien de triangles sont dessinés par le script ?
3. Quelle est la nature des triangles dessinés ?
4. Quelle est la longueur en pas des triangles tracés.
5. Tracer le dessin obtenu en prenant 1 cm pour 20 pas.
6. On souhaite tracer à la place des hexagones réguliers. Que faut-il modifier ?

Exercice 10 — Analyse générale de programme

Extrait du CRPE 2024 Groupement 1, Sciences et Technologie

Un enseignant propose une séance pédagogique à ses élèves de CM2 pour simuler la fonction suivante : « *Afficher un signal lumineux lors d'une touche entre l'extrémité du fleuret de l'escrimeuse et le corps de son adversaire* ». Il utilise une carte « Micro :bit V2 » qui se programme à l'aide d'une interface de programmation par blocs. Cette carte programmable comporte un afficheur de 25 diodes électroluminescentes LED (matrice de 5x5 LED), deux boutons A et B, différents capteurs et actionneurs intégrés, des broches de connexion.

L'objectif de la séance est de réaliser un programme permettant d'afficher un symbole lumineux « ✓ » (forme d'encoche) lorsque le bouton A de la carte est appuyé, simulant ainsi la touche du fleuret.

Voici les programmes réalisés par deux élèves au cours de cette séance afin de simuler l'affichage d'une touche et le programme attendu.

<pre> toujours si bouton A ▼ est pressé alors montrer l'icône [icône] pause (ms) 1000 sinon effacer l'écran pause (ms) 1000 + </pre>	<pre> au démarrage si bouton A ▼ est pressé alors montrer l'icône [icône] pause (ms) 1000 + si bouton B ▼ est pressé alors montrer l'icône [icône] pause (ms) 1000 + </pre>	<pre> toujours si bouton A ▼ est pressé alors montrer l'icône [icône] pause (ms) 1000 + si bouton B ▼ est pressé alors montrer l'icône [icône] pause (ms) 1000 + </pre>
Programme attendu	Élève n°1	Élève n°2

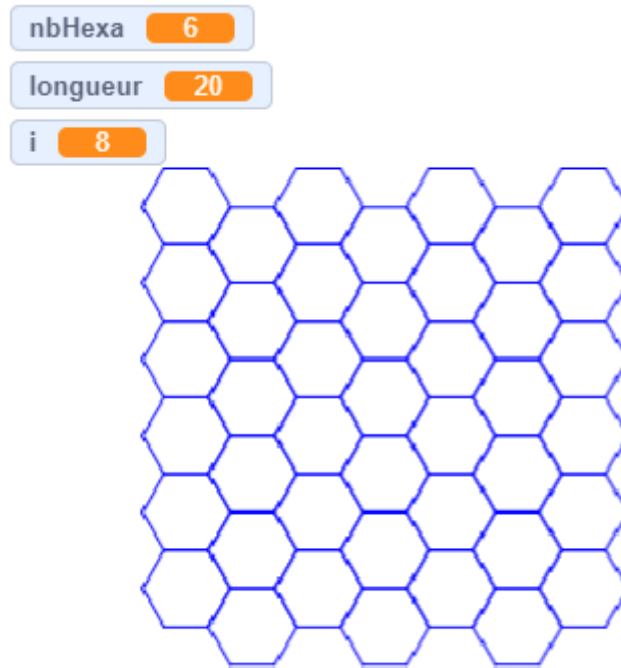
Précisions supplémentaires à apporter ici :

- le bloc **Toujours** est une boucle qui ne s'arrête jamais. Si le programme se termine, alors la carte est **mise à l'arrêt** ;
- Le bloc **Montrer l'icône** allume les LED correspondantes sur la carte. L'état des LED **ne change pas tant qu'une autre instruction modifiant l'affichage n'est pas appelée**.

1. Traduire le programme attendu en langage naturel.
2. Expliquer pour chaque production d'élève ce qu'il se passe en appuyant sur A.
Pour répondre à cette question, on pourra vérifier la présence d'une boucle infinie, et l'état du programme lorsqu'on a appuyé une fois sur le bouton et recommencé la boucle.

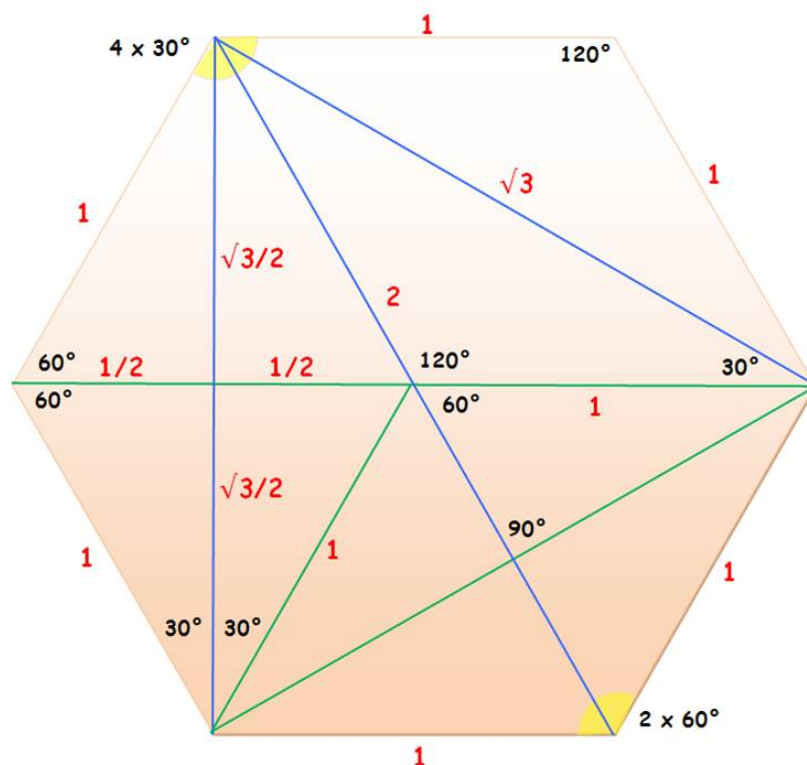
Exercice 11 — Projet de pavage

On souhaite réaliser le pavage ci-dessous avec Scratch :



En décomposant les opérations à réaliser, on comprend vite qu'il faut une procédure pour :

- construire un hexagone régulier d'une longueur donnée ;
- construire une série d'hexagones ;
- construire un pavage.



Pour des raisons pédagogiques, nous détaillons les deux premières procédures.

1. Pour construire un hexagone de côté l , on commence par se placer sur un sommet. On choisit le sommet en haut à gauche, donc on va s'orienter à droite. On va donc répéter 6 fois les opérations "avancer de l ", puis "tourner de 60 degrés dans le sens horaire".
 - (a) on voit que les angles d'un hexagone sont de 120 degrés, alors pourquoi est-ce qu'on tourne de 60 degrés ? Répondre puis illustrer par un schéma.
 - (b) Sur quel sommet finit-on après avoir tracé l'hexagone ?
2. Pour passer d'un hexagone au suivant, on va descendre de la hauteur de l'hexagone, la diagonale qui a pour longueur $\sqrt{3}$ dans un hexagone de côté 1, et $l\sqrt{3}$ dans un hexagone de côté l .
3. Une fois la colonne finie, comment remonter ? Tracer un schéma avec 4 hexagones consécutifs, puis indiquer de combien on s'est déplacé sur l'axe y

Exercice 12 — Jeu de hasard DNB Amérique du Nord, 2022

On a créé un jeu de hasard à l'aide d'un logiciel de programmation. Lorsqu'on appuie sur le drapeau, le lutin dessine trois motifs côte à côte. Chaque motif est dessiné aléatoirement : soit c'est une croix, soit c'est un rectangle. Le joueur gagne si l'affichage obtenu comporte trois motifs identiques. Au lancement du programme, le lutin est orienté horizontalement vers la droite.

Programme principal		Bloc « rectangle »
Numéro de ligne ↓	<pre> 1 quand [drapeau] est cliqué 2 effacer tout 3 aller à x: -110 y: 0 4 répéter 3 fois 5 si [nombre aléatoire entre 1 et 2 = 1] alors 6 croix 7 sinon 8 rectangle 9 fin si 10 avancer de 100 pas 11 </pre>	<pre> définir rectangle stylo en position d'écriture Répéter 2 fois avancer de 60 pas tourner de 90 degrés avancer de 80 pas tourner de 90 degrés relever le stylo </pre>
		Bloc « croix » Le script n'est pas donné.
Explication de l'instruction « nombre aléatoire entre... » sur un exemple : nombre aléatoire entre 1 et 4 renvoie un nombre au hasard parmi 1, 2, 3 et 4.		

1. En prenant pour échelle 1 cm pour 20 pas, représenter le motif obtenu par le bloc « rectangle ».
2. Quelle est la distance en pas entre deux rectangles consécutifs ? Détailler la réponse.
3. Quelle est la probabilité que le premier motif dessiné par le lutin soit une croix ?

4. Dessiner à main levée tous les affichages différents que l'on pourrait obtenir avec le programme principal.
5. On suppose que la génération aléatoire des nombres est uniforme (équiprobabilité). Quelle est la probabilité de gagner ?
On souhaite à présent qu'il y ait deux fois plus de chances d'obtenir un rectangle qu'une croix.
6. Modifier l'instruction de la ligne 5 du programme principal en conséquence.
7. Quelle est la nouvelle probabilité de gagner ?